Gimel Foundation

Gimel Foundation gGmbH i.G.
GiFo-Request for Comments: 0117
Obsoletes: -
Category: Standards Track
Version: 1.2

G. Wehberg, S. Raghuthaman and the Architecture Working Group of Gimel Foundation
8. April 2026

GAuth: Power Enforcement Point

Abstract

This specification defines the Policy Enforcement Point (PEP) interface for the Gimel Authorization (GAuth) protocol. It determines the PEP interface with two layers:

- **Normative Core** - A protocol-level PEP Interface Contract defining the input schema (enforcement request), output schema (enforcement decision), error response schema, evaluation semantics (how each PoA field maps to an enforcement check with defined order and precedence), interaction with RFC 0116 §10.2 validation modes, and an HTTP binding for remote PEP deployments.
- **Non-Normative Guidance** - Four deployment patterns for PEP integration (Middleware/Interceptor, Sidecar/Proxy, SDK/Client Library, MCP Compliance Proxy), with a trade-off matrix.

Together, these enable any party that receives PoA credentials — whether it issued them (via an RFC 0116 Type A adapter), holds them for presentation to humans or cross-platform delegation (via a Type B adapter — a future interoperability path for human-facing PoA disclosure, and EUDI regulatory compliance), or simply accepts them as a relying party — to enforce those credentials consistently and in an interoperable way, based on a published specification rather than implementation-specific knowledge.

This RFC is a Gimel Foundation contribution paper intended for open-source publication under Apache License 2.0. Proprietary platform extensions, i.e. so-called Exclusions (as per Legal Terms of Gimel Foundation) like AI-based governance, DNA-based identity and PQC associated, Web3 integration, are outside the scope of this specification and are not covered by the Apache 2.0 license (see Section 2 of this Request of Comment).

Status of This Memo

This is a Gimel Foundation Standards Track document.

This document is a product of the Gimel Foundation (GiFo). It represents the current consensus of the Gimel Foundation community. It has performed review and has been approved for publication.

Information about the status of this document, any errata, and how to provide feedback on it may be obtained at https://gimelfoundation.com or https://github.com/Gimel-Foundation.

Legal notice

Copyright (c) 2026 Gimel Foundation and the persons identified as the document authors. All rights are reserved.

This document is subject to the Gimel Foundation's Legal Provisions Relating to GiFo Documents (see http://GimelFoundation.com or https://github.com/Gimel-Foundation) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Code Components extracted from this document must include License text as described in Section 4. of the GiFo Legal Provisions Relating to Gimel Foundation Documents and are provided without warranty as described in the Provisions and its respective license conditions.

The distinguished GAuth standard as well as its PEP are protected by copy right and patent law (patent pending). GAuth is an open-source standard, which can be based on OAuth, OpenID Connect and MCP. GAuth as well as PEP must not use Exclusions (see Scope under Section 2 of this Request for Comment), which are subject to separate license conditions and are also protected by copy right as well as patent law (patent pending).

Implementations of GAuth as well as these PEP standards must reference to the Apache 2.0 license of OAuth and OpenID Connect as well as to the MIT license of MCP in line with its license conditions, as building blocks. Copyrights and licenses of these building blocks apply accordingly.

Implementations of GAuth as well as these PEP standards must being licensed with Apache 2.0, granted by Gimel Foundation, and must not integrate Exclusions (as per Scope statement under Section 2 of this Request for Comment). Defined Exclusions of GAuth as well as of these PEP standards must refer to separate license conditions.

Notational Conventions

The key words "Must", "Must Not", "Required", "Shall", "Shall Not", "Should", "Should Not", "Recommended", "May", and "Optional" in the following specification are to be interpreted as described in IETF`s RFC 2119.

* * *

Table of Content

1. Introduction
2. Legal Framework & Licensing
3. Architectural Overview
4. PEP Interface Contract - Enforcement Request
5. PEP Interface Contract - Enforcement Decision
6. PEP Interface Contract — Error Response
7. PEP Interface Contract — Method Signatures
8. PEP Interface Contract — HTTP Binding
9. Evaluation Semantics
10. Stateless vs. Stateful Enforcement
11. Delegation Chain Enforcement
12. Interface Versioning
13. Deployment Patterns (Non-Normative)
14. Conformance Requirements
15. Security Considerations

Appendices:

A: Reference Implementation Pseudocode
B: Deployment Pattern Tradeoff Matrix
C: PEP Claims Registry
D: Example Verb Namespaces
E: Open Questions
F: Reference Documents

1. INTRODUCTION

1.1 Purpose

GAuth defines a comprehensive authorization protocol for governing agent-based software systems. The protocol suite — RFCs 0110, 0111, 0115, and 0116 — specifies how PoA credentials are structured, issued, transported, and validated. However, it does not specify in detail how those credentials are "enforced" — i.e., how a runtime system evaluates an incoming agent action against the PoA to produce a permit/deny/constrain decision.

Any service that receives a PoA credential — whether it is the issuing platform that originally issued it, a third-party platform that accepts GAuth tokens as a relying party, or (in future deployments) a credential wallet that holds PoA credentials for human-facing disclosure or cross-platform delegation — needs the specification for how to evaluate that credential against incoming agent actions.

This RFC provides that specification.

1.2 Design Principles

- **Interface-first** - The PEP is defined as an interface contract (input/output schemas + evaluation semantics), not as an implementation. Any language, framework, or runtime can implement the PEP.
- **Fail-closed** - If the PEP cannot reach a definitive decision (missing data, evaluation error, timeout), the result is DENY. There is no fail-open mode.
- **Deterministic** - Given the same enforcement request and the same PoA, the PEP MUST produce the same decision. Non-deterministic enrichment (AI-assisted verdicts, probabilistic scoring) operates in a separate layer and is not part of the normative PEP contract.
- **Composable** - The PEP may be augmented with pre-enforcement (threat scanning, compliance gating) and post-enforcement (audit recording, metering) steps. The PEP contract covers only the authorization decision itself.
- ***Extendable*** *- Proprietary features can additionally cover so called implied authority instances, where explicit authority is not available for a rule-based approval, but such proprietary features are out of scope of this RFC0117, and excluded from the Apache 2.0 license (Exclusions – see also Section 2 of this Request for Comments: Out-of-Scope).*

1.3 Terminology

PEP (Policy Enforcement Point): A runtime component that intercepts agent actions and evaluates them against the PoA to produce a permit/deny/constrain decision.

PDP (Policy Decision Point): The logical component that evaluates policy rules. In many GAuth deployments, the PEP and PDP are co-located - the PEP contains the decision logic. However, some deployments MAY separate them (e.g., a sidecar PEP that delegates decisions to a remote PDP). This specification defines the combined PEP+PDP contract; deployments that separate them MUST ensure the PDP implements the evaluation semantics of Section 9.

Enforcement Request: A structured input describing the agent action being evaluated.

Enforcement Decision: A structured output describing whether the action is permitted, denied, or constrained.

PoA Snapshot: The frozen PoA credential as it exists in the JWT or VC at presentation time.

Live PoA: The current PoA state as stored by the issuer, reflecting real-time budget, status, and permission changes.

1.4 Relationship to Prior RFCs

```
GiFo-RFC 0110  ──→  Protocol Engine (transport, routing, service mesh)
     │
GiFo-RFC 0111  ──→  Authorization Framework (flows, roles, trust model)
     │
GiFo-RFC 0115  ──→  Three-Layer Capability Model (Vollmacht conceptual schema)
     │                └─ Governance profile definitions and ceiling tables
     │
GiFo-RFC 0116  ──→  Interoperability Layer (token format, schemas, adapters)
     │                ├─ §4: PoA Credential Schema
```

```
     │                    ├─ §10.2: Validation Modes (stateless /
stateful)
     │                    ├─ §12.2: Error Response Schema
     │                    └─ §13: Conformance Requirements
     │
GiFo-RFC 0117  ──→  Policy Enforcement Point (THIS DOCUMENT)
  (this doc)          ├─ §4-8: PEP Interface Contract (request,
decision, errors, methods, HTTP binding)
                      ├─ §9: Evaluation Semantics (PoA field →
check mapping)
                      ├─ §10: Stateless vs. Stateful
Enforcement
                      ├─ §11: Delegation Chain Enforcement
                      ├─ §12: Interface Versioning
                      └─ §13: Deployment Patterns (non-
normative)
```

1.5 Scope (technical view)

In technical Scope:

- PEP interface contract (input/output schemas, error responses, method signatures, HTTP binding)

- Evaluation semantics (field-to-check mapping, ordering, precedence)

- Interaction with RFC 0116 validation modes

- Delegation chain enforcement

- Interface versioning

- Deployment patterns (non-normative guidance)

Out of technical Scope:

- Management API for PoA CRUD operations (future RFC 0118)

- Proprietary enforcement extensions (Type C/D adapters — outside the scope of this open specification)

- AI-assisted verdict enrichment (operates above the PEP layer; platform-specific)

- Billing and metering of PEP operations (internal platform concern)

- Governance profile ceiling definitions (defined in RFC 0115)

2. LEGAL FRAMEWORK & LICENSING

2.1 Apache License 2.0 Scope

GiFo-RFC 0117 is published under the Apache License, Version 2.0. The PEP interface contract, evaluation semantics, and reference implementation pseudocode are fully open.

In Scope (Open Source — Apache 2.0):

- **PEP Interface Contract** (Sections 4-8): Input/output schemas, error responses, method signatures, HTTP binding.
- **Evaluation Semantics** (Section 9): PoA field-to-check mapping, ordering, precedence.
- **Deployment Patterns** (Section 13): Non-normative guidance for PEP integration.
- **Reference Implementation** (Appendix A): Language-agnostic pseudocode.

Out of Scope (Not part of this specification – license view):

- **PEP extensions**: AI-assisted verdicts, threat scoring, compliance gating — these are platform-specific and not part of the open PEP contract.
- **Exclusions**: Exclusions as per RFC0116 apply.,

The Legal Terms of Gimel Foundation apply.

3. ARCHITECTURAL OVERVIEW

3.1 PEP in the GAuth Architecture

The PEP sits between the requesting agent and the target resource. It intercepts every action and evaluates the agent's PoA credential before the action reaches the resource.

```
┌──────────────────────────────────────────────────────────────────┐
│  Agent / Service (carrying PoA credential)                       │
│                                                                  │
│  1. Agent requests action (e.g., file.modify, command.run)       │
```

```
│  2. Agent presents PoA (JWT or VC)                                  │
└──────────────┬──────────────────────────────────────────────────────┘
               │ Enforcement Request
               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  POLICY ENFORCEMENT POINT (PEP) — RFC 0117                           │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  1. Credential Validation (RFC 0116 §10.2)                   │   │
│  │     • Signature, temporal validity, audience, checksum       │   │
│  ├──────────────────────────────────────────────────────────────┤   │
│  │  2. PoA Enforcement (RFC 0117 §9 — this document)            │   │
│  │     • Temporal → Profile → Phase → Sector → Region →        │   │
│  │       Path → Verb → Constraints → Platform →                 │   │
│  │       Transaction → Decision → Budget → Session →            │   │
│  │       Approval → Delegation Chain                            │   │
│  ├──────────────────────────────────────────────────────────────┤   │
│  │  3. Decision: PERMIT / DENY / CONSTRAIN                      │   │
│  │     + enforced constraints, violations, audit record         │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────┬──────────────────────────────────────────────────────┘
               │ Enforcement Decision
               ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Target Resource / Service                                           │
│  (action proceeds only if PERMIT or CONSTRAIN)                       │
└──────────────────────────────────────────────────────────────────────┘
```

3.2 PEP vs. Pre-/Post-Enforcement Layers

The PEP is the “authorization decision” layer. It answers one question: "Does this PoA permit this action?" Additional enforcement layers may surround the PEP but are not part of the PEP contract:

- **Pre-enforcement**: S2S authentication, threat scanning, compliance gating | Runs *before* the PEP. Rejects unauthenticated or malicious requests before PEP evaluation.
- **PEP** (this specification): PoA authorization decision - The normative contract defined here.
- **Post-enforcement**: Audit logging, usage recording, billing, metering - Runs *after* the PEP. Records the decision and its consequences.

Implementations MAY combine pre-enforcement, PEP, and post-enforcement into a single pipeline (as is common in middleware deployments). The logical separation remains: the PEP evaluates the PoA, and the pre-/post-enforcement layers handle everything else.

3.3 PEP and Credential Validation

The PEP MUST validate the PoA credential before evaluating it. Credential validation follows RFC 0116 §10.2:

- **Stateless validation** (§10.2.1): Signature, temporal, audience, scope_checksum. Required for all PEP evaluations.

- **Stateful validation** (§10.2.2): Mandate status, budget, full permission lookup. Required when the PEP is in stateful mode (Section 10).

Credential validation is a prerequisite for PEP evaluation, not a PEP function itself. The PEP MAY delegate credential validation to the OAuth Engine Adapter's `introspectToken()` or perform it directly using JWKS.

4. PEP INTERFACE CONTRACT — ENFORCEMENT REQUEST

4.1 EnforcementRequest Schema

The enforcement request describes the agent action being evaluated. All PEP implementations MUST accept this schema.

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
```

```
  "$id":
"https://gimelfoundation.com/schemas/pep/v1.2/enforcement-
request.json",

  "title": "GAuth PEP Enforcement Request",

  "type": "object",

  "required": ["action", "agent", "credential", "request_id",
"timestamp"],

  "properties": {

    "request_id": {

      "type": "string",

      "description": "Unique correlation ID for this
enforcement request. Used for audit trail linkage."

    },

    "timestamp": {

      "type": "string",

      "format": "date-time",

      "description": "ISO 8601 timestamp of the enforcement
request."

    },

    "action": {

      "$ref": "#/$defs/ActionDescriptor",

      "description": "The agent action being evaluated."

    },

    "agent": {

      "$ref": "#/$defs/AgentIdentity",

      "description": "The identity of the requesting agent."

    },

    "credential": {

      "$ref": "#/$defs/CredentialReference",

      "description": "The PoA credential presented by the
agent."

    },
```

```
    "context": {
      "$ref": "#/$defs/EnforcementContext",
      "description": "Optional context for stateful
enforcement."
    }
  },
  "$defs": {
    "ActionDescriptor": {
      "type": "object",
      "required": ["verb", "resource"],
      "properties": {
        "verb": {
          "type": "string",
          "description": "The action verb being requested.
MUST use the URN format
urn:gauth:verb:{domain}:{category}:{action} and match a verb
registered in the PoA's core_verbs namespace. See Appendix D
for example verb namespaces."
        },
        "resource": {
          "type": "string",
          "description": "The resource being acted upon (e.g.,
file path, database table, deployment target). Interpretation
is verb-specific."
        },
        "resource_type": {
          "type": "string",
          "description": "Classification of the resource type.
Used for path validation and platform permission mapping.
Implementations define their own resource type vocabulary;
common values include 'file', 'directory', 'database',
'deployment', 'shell', 'package', 'secret', 'agent', 'api'."
        },
        "parameters": {
```

```
          "type": "object",

          "additionalProperties": true,

          "description": "Verb-specific parameters. Structure
depends on the verb and the implementation's domain. The only
normative parameter is 'amount_cents' (integer, >= 0) when the
action has a cost estimate for budget enforcement. All other
parameters are implementation-defined.",

          "properties": {

            "amount_cents": {

              "type": "integer",

              "minimum": 0,

              "description": "Cost estimate in cents for
budget enforcement. This is the only normative parameter."

            }

          }

        },

        "sector": {

          "type": "string",

          "description": "Business sector context for sector-
restricted PoAs. Implementations SHOULD use NAICS codes or
equivalent standardized sector identifiers."

        },

        "region": {

          "type": "string",

          "description": "Geographic region context for
region-restricted PoAs. Implementations SHOULD use ISO 3166-1
alpha-2 country codes or supranational identifiers (e.g.,
'EU')."

        },

        "transaction_type": {

          "type": "string",

          "description": "Transaction classification for PoAs
with transaction matrix restrictions."

        },
```

```
        "decision_type": {
          "type": "string",
          "description": "Decision classification for PoAs
with decision-type restrictions."
        }
      }
    },
    "AgentIdentity": {
      "type": "object",
      "required": ["agent_id"],
      "properties": {
        "agent_id": {
          "type": "string",
          "description": "External agent identifier (matches
PoA subject)."
        },
        "service": {
          "type": "string",
          "description": "Calling service identifier."
        },
        "session_id": {
          "type": "string",
          "description": "Current session identifier. Used for
session limit enforcement."
        },
        "did": {
          "type": "string",
          "description": "DID of the agent, if available. Used
for delegation chain verification."
        }
      }
```

```
    },
    "CredentialReference": {
      "type": "object",
      "required": ["format"],
      "properties": {
        "format": {
          "type": "string",
          "enum": ["jwt", "w3c_vc", "sd-jwt"],
          "description": "Credential format."
        },
        "token": {
          "type": "string",
          "description": "Raw credential (JWT string, VC JSON,
or SD-JWT). Required if the PEP performs its own credential
validation."
        },
        "mandate_id": {
          "type": "string",
          "description": "Mandate ID for stateful lookup. Used
when the PEP delegates credential validation to the issuer."
        },
        "poa_snapshot": {
          "type": "object",
          "description": "Pre-parsed PoA claims from the
credential. Provided when credential validation has already
been performed upstream (e.g., by an S2S auth middleware)."
        }
      }
    },
    "EnforcementContext": {
      "type": "object",
```

```
      "description": "Optional context for stateful
enforcement or enrichment.",
      "properties": {
        "session_state": {
          "type": "object",
          "properties": {
            "tool_calls_used": { "type": "integer", "minimum":
0 },
            "lines_committed": { "type": "integer", "minimum":
0 },
            "session_started_at": { "type": "string",
"format": "date-time" },
            "session_cost_cents": { "type": "integer",
"minimum": 0 }
          }
        },
        "live_mandate_state": {
          "type": "object",
          "description": "Live mandate state fetched from the
issuer. Used in stateful enforcement mode.",
          "properties": {
            "status": { "type": "string", "enum": ["active",
"expired", "revoked", "budget_exceeded", "superseded"] },
            "budget_remaining_cents": { "type": "integer",
"minimum": 0 },
            "tool_permissions": { "type": "object" },
            "platform_permissions": { "type": "object" }
          }
        }
      }
    }
  }
}
```

```
```

4.2 Action Verb Namespace

Action verbs in the enforcement request MUST use the GAuth verb URN format:

```
```

```
urn:gauth:verb:{domain}:{category}:{action}
```

Where:

- `{domain}` — The service or platform domain that defines the verb (e.g., a vendor name, a service type, or a generic domain like `core`)

- `{category}` — The resource category the verb operates on (e.g., `file`, `command`, `agent`, `database`)

- `{action}` — The specific action (e.g., `create`, `modify`, `delete`, `run`, `delegate`)

Custom verbs MUST be registered in the PoA's `core_verbs` object to be enforceable. Implementations define their own verb namespaces within this format. See Appendix D for example verb namespaces.

5. PEP INTERFACE CONTRACT — ENFORCEMENT DECISION

5.1 EnforcementDecision Schema

The enforcement decision describes the PEP's ruling on the enforcement request. All PEP implementations MUST return this schema.

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gimelfoundation.com/schemas/pep/v1.2/enforcement-decision.json",
  "title": "GAuth PEP Enforcement Decision",
  "type": "object",
```

```
  "required": ["request_id", "decision", "timestamp",
"enforcement_mode", "checks"],
  "properties": {
    "request_id": {
      "type": "string",
      "description": "Echo of the enforcement request's
correlation ID."
    },
    "decision": {
      "type": "string",
      "enum": ["PERMIT", "DENY", "CONSTRAIN"],
      "description": "The enforcement decision."
    },
    "timestamp": {
      "type": "string",
      "format": "date-time",
      "description": "ISO 8601 timestamp of the decision."
    },
    "enforcement_mode": {
      "type": "string",
      "enum": ["stateless", "stateful"],
      "description": "Which enforcement mode was used for this
decision."
    },
    "checks": {
      "type": "array",
      "items": { "$ref": "#/$defs/CheckResult" },
      "description": "Ordered list of enforcement checks
performed."
    },
    "enforced_constraints": {
```

```
      "type": "array",
      "items": { "$ref": "#/$defs/EnforcedConstraint" },
      "description": "Constraints applied to the action when
decision is CONSTRAIN. Empty for PERMIT and DENY."
    },
    "violations": {
      "type": "array",
      "items": { "$ref": "#/$defs/Violation" },
      "description": "Policy violations detected. Non-empty
when decision is DENY."
    },
    "audit": {
      "$ref": "#/$defs/AuditRecord",
      "description": "Audit information for logging and
traceability."
    }
  },
  "$defs": {
    "CheckResult": {
      "type": "object",
      "required": ["check_id", "check_name", "result"],
      "properties": {
        "check_id": {
          "type": "string",
          "description": "Unique identifier for this check
(e.g., 'CHK-01-TEMPORAL')."
        },
        "check_name": {
          "type": "string",
          "description": "Human-readable check name (e.g.,
'Temporal Validity')."
        },
```

```
        "result": {
          "type": "string",
          "enum": ["pass", "fail", "skip", "constrain"],
          "description": "Check outcome. 'skip' when the check
is not applicable (e.g., no sector restrictions in the PoA)."
        },
        "detail": {
          "type": "string",
          "description": "Human-readable detail about the
check outcome."
        }
      }
    },
    "EnforcedConstraint": {
      "type": "object",
      "required": ["constraint_type", "check_id", "requested",
"enforced"],
      "properties": {
        "constraint_type": {
          "type": "string",
          "description": "Machine-readable constraint
identifier (e.g., 'path_restricted', 'budget_capped')."
        },
        "check_id": {
          "type": "string",
          "description": "The check_id that produced this
constraint."
        },
        "requested": {
          "description": "What the action originally
requested."
        },
```

```
        "enforced": {
          "description": "The constrained value the action
must respect."
        }
      }
    },
    "Violation": {
      "type": "object",
      "required": ["code", "message", "check_id", "severity"],
      "properties": {
        "code": {
          "type": "string",
          "description": "Machine-readable violation code from
the violation code registry (Section 5.3)."
        },
        "message": {
          "type": "string",
          "description": "Human-readable violation
description."
        },
        "check_id": {
          "type": "string",
          "description": "The check_id that produced this
violation."
        },
        "severity": {
          "type": "string",
          "enum": ["error", "warning"],
          "description": "Violation severity. 'error'
contributes to a DENY decision; 'warning' is logged in the
audit record but does not affect the decision."
        }
```

```
      }
    },
    "AuditRecord": {
      "type": "object",
      "required": ["processing_time_ms", "pep_version"],
      "properties": {
        "processing_time_ms": {
          "type": "number",
          "description": "Total PEP processing time in
milliseconds."
        },
        "pep_version": {
          "type": "string",
          "description": "Implementation version of the PEP
(for audit and debugging)."
        },
        "pep_interface_version": {
          "type": "string",
          "description": "RFC 0117 interface version
implemented (e.g., '1.1'). Enables clients to understand which
checks and schemas the PEP supports."
        },
        "credential_jti": {
          "type": "string",
          "description": "JWT ID from the presented credential
(for audit trail linkage). MUST NOT include the full
credential payload."
        },
        "mandate_id": {
          "type": "string",
          "description": "Mandate ID (if applicable and
available)."
```

```
        },
        "agent_id": {
          "type": "string",
          "description": "Agent identity from the enforcement
request."
        },
        "action_verb": {
          "type": "string",
          "description": "Action verb from the enforcement
request."
        },
        "action_resource": {
          "type": "string",
          "description": "Action resource from the enforcement
request."
        },
        "checks_performed": {
          "type": "integer",
          "description": "Number of checks performed."
        },
        "checks_passed": {
          "type": "integer",
          "description": "Number of checks that passed."
        },
        "checks_failed": {
          "type": "integer",
          "description": "Number of checks that failed."
        }
      }
    }
  }
```

```
}
```

5.2 Decision Semantics

- PERMIT: All applicable enforcement checks passed. The action is authorized. | Proceed to target resource |
- DENY: One or more enforcement checks failed with `error` severity. The action is not authorized. | Reject the action. Return violations to the caller. |
- CONSTRAIN: The action is authorized but with constraints applied. The PoA permits the action within boundaries (e.g., path restrictions, budget caps, delegation depth limits). | Proceed to target resource with constraints. The caller and target MUST respect the enforced constraints. |

Decision precedence:

1. If any check produces `result: "fail"` AND the corresponding violation has `severity: "error"`, the decision is **DENY** regardless of other checks.

2. Violations with `severity: "warning"` are included in the `violations` array for audit purposes but do **not** contribute to the decision. A check that produces a warning-severity violation is treated as `pass` for decision purposes.

3. If all checks pass (no error-severity failures) but one or more produce `result: "constrain"`, the decision is **CONSTRAIN**.

4. If all applicable checks pass with no constraints and no error-severity violations, the decision is **PERMIT**.

5.3 Violation Code Registry

```
| Code | Check | Description |
|------|-------|-------------|
| `CREDENTIAL_INVALID` | CHK-01 | PoA credential failed signature or integrity verification |
| `CREDENTIAL_EXPIRED` | CHK-02 | PoA credential has expired (`exp` < current time) |
| `CREDENTIAL_REVOKED` | CHK-02 | PoA credential / mandate has been revoked |
```

| `CREDENTIAL_SUPERSEDED` | CHK-02 | Mandate has been superseded by a newer mandate |

| `AGENT_MISMATCH` | CHK-02 | Agent identity does not match the PoA subject |

| `PROFILE_CEILING_EXCEEDED` | CHK-03 | Action exceeds governance profile ceiling |

| `PHASE_MISMATCH` | CHK-04 | Action not permitted in the current phase |

| `SECTOR_MISMATCH` | CHK-05 | Action sector is not in the PoA's allowed sectors |

| `REGION_MISMATCH` | CHK-06 | Action region is not in the PoA's allowed regions |

| `PATH_DENIED` | CHK-07 | Resource path is explicitly denied |

| `PATH_NOT_ALLOWED` | CHK-07 | Resource path is not in the allowed paths set |

| `VERB_NOT_ALLOWED` | CHK-08 | Action verb is not permitted by the PoA |

| `CONSTRAINT_VIOLATED` | CHK-09 | Action violates a verb-level constraint |

| `PLATFORM_PERMISSION_DENIED` | CHK-10 | Agent lacks the required platform permission |

| `TRANSACTION_NOT_ALLOWED` | CHK-11 | Transaction type is not permitted or fails matrix check |

| `DECISION_NOT_ALLOWED` | CHK-12 | Decision type is not permitted by the PoA |

| `BUDGET_EXCEEDED` | CHK-13 | Action cost would exceed the remaining budget |

| `BUDGET_EXHAUSTED` | CHK-13 | Budget is fully exhausted (remaining = 0) |

| `SESSION_LIMIT_EXCEEDED` | CHK-14 | Session limit reached (tool calls, duration, or lines) |

| `APPROVAL_REQUIRED` | CHK-15 | Action requires human approval that has not been granted |

| `DELEGATION_DEPTH_EXCEEDED` | CHK-16 | Delegation chain exceeds maximum depth |

| `DELEGATION_SCOPE_EXCEEDED` | CHK-16 | Delegated scope exceeds delegator's scope |

6. PEP INTERFACE CONTRACT — ERROR RESPONSE

6.1 Error Response Schema

When the PEP cannot process an enforcement request (as opposed to denying it on policy grounds), it MUST return an error response conforming to this schema. This aligns with the error response format defined in RFC 0116 §12.2 (TG-02).

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://gimelfoundation.com/schemas/pep/v1.2/enforcement-error.json",
  "title": "GAuth PEP Error Response",
  "type": "object",
  "required": ["error_code", "message", "timestamp"],
  "properties": {
    "error_code": {
      "type": "string",
      "enum": ["PEP_INTERNAL_ERROR", "INVALID_REQUEST", "CREDENTIAL_PARSE_ERROR", "ISSUER_UNREACHABLE", "EVALUATION_TIMEOUT"],
      "description": "Machine-readable error code."
    },
    "message": {
      "type": "string",
      "description": "Human-readable error description."
    },
    "timestamp": {
      "type": "string",
      "format": "date-time",
```

```
      "description": "ISO 8601 timestamp of the error."
    },
    "request_id": {
      "type": "string",
      "description": "Echo of the enforcement request's
correlation ID, if available."
    },
    "detail": {
      "type": "object",
      "description": "Additional error context. Structure
depends on the error_code.",
      "properties": {
        "failed_field": { "type": "string", "description":
"The request field that caused the error (for
INVALID_REQUEST)." },
        "issuer_url": { "type": "string", "description": "The
issuer URL that was unreachable (for ISSUER_UNREACHABLE)." },
        "timeout_ms": { "type": "number", "description": "The
timeout threshold that was exceeded (for EVALUATION_TIMEOUT)."
}
      }
    }
  }
}
```

6.2 Error Code Semantics

```
| Error Code | HTTP Status | Meaning |
|-----------|-------------|---------|
| `INVALID_REQUEST` | 400 | The enforcement request is
malformed or missing required fields |
| `CREDENTIAL_PARSE_ERROR` | 400 | The credential could not be
parsed (invalid JWT structure, malformed VC) |
```

```
| `ISSUER_UNREACHABLE` | 502 | Stateful enforcement failed
because the issuer could not be reached |

| `EVALUATION_TIMEOUT` | 504 | The evaluation pipeline
exceeded the configured timeout |

| `PEP_INTERNAL_ERROR` | 500 | An unexpected internal error
occurred during evaluation |
```

Important: Error responses are distinct from DENY decisions. A DENY decision means the PEP successfully evaluated the request and determined it is not authorized. An error response means the PEP could not complete evaluation. Per the fail-closed principle (§1.2), callers MUST treat error responses as equivalent to DENY — the action MUST NOT proceed.

7. PEP INTERFACE CONTRACT — METHOD SIGNATURES

7.1 enforceAction

The primary PEP method. Evaluates a single agent action against the presented PoA credential.

```
enforceAction(request: EnforcementRequest) →
EnforcementDecision | EnforcementError


  Inputs:  EnforcementRequest (Section 4.1)

  Outputs: EnforcementDecision (Section 5.1) or
EnforcementError (Section 6.1)


  Behavior:

    1. Validate credential (RFC 0116 §10.2)

    2. Execute evaluation pipeline (Section 9)

    3. Return decision with checks, constraints, violations,
audit


  Latency targets (non-normative):

    Stateless mode: < 5ms
```

```
    Stateful mode:  < 100ms
```

7.2 batchEnforce

Evaluates multiple actions in a single call. Useful for pre-flight checks (e.g., "Can this agent perform all of these actions?") and transaction-like operations where multiple actions must all succeed or all fail.

```
batchEnforce(requests: EnforcementRequest[], mode:
"all_or_nothing" | "independent") → BatchDecision


  Inputs:

    requests: Array of EnforcementRequest

    mode:

      "all_or_nothing" — all requests must be
PERMIT/CONSTRAIN, or all are DENY

      "independent"    — each request evaluated independently


  Outputs:

    BatchDecision {

      overall_decision: PERMIT | DENY | CONSTRAIN

      decisions: EnforcementDecision[]  // one per request,
same order

    }


  Behavior:

    For "all_or_nothing": if any individual decision is DENY,
all are DENY.

    For "independent": each request produces its own decision.
```

7.3 getEnforcementPolicy

Returns a summary of the enforcement policy for a given PoA. Enables callers to pre-inspect what actions are available before attempting them.

```
getEnforcementPolicy(credential: CredentialReference) →
EnforcementPolicy


  Inputs:  CredentialReference (token or mandate_id)
  Outputs:
    EnforcementPolicy {
      governance_profile: string
      phase: string
      allowed_verbs: string[]
      denied_paths: string[]
      allowed_paths: string[]
      permissions: object              // Implementation-
defined permission summary.
                                       // Structure depends on
the platform's Layer 3
                                       // permission model.
Callers SHOULD NOT rely on
                                       // specific keys — use
this for display/pre-flight
                                       // purposes only.
      budget: {
        total_cents: number
        remaining_cents: number
      }
      session_limits: object           // Implementation-
defined session limit summary.
      approval_mode: string
      delegation: {
```

```
        allowed: boolean

        max_depth: number

      }

    }


  Errors:  CREDENTIAL_PARSE_ERROR (400), CREDENTIAL_INVALID →
DENY, PEP_INTERNAL_ERROR (500)
```

8. PEP INTERFACE CONTRACT — HTTP BINDING

8.1 Purpose

For PEP deployments where the PEP runs as a separate process (Patterns B and D in Section 13), an HTTP binding is required so that services can submit enforcement requests over the network. This section defines the normative HTTP API surface.

8.2 Endpoints

Method	Path	Request Body	Response Body	Description
`POST`	`/gauth/pep/v1/enforce`	`EnforcementRequest`	`EnforcementDecision` or `EnforcementError`	Single action enforcement
`POST`	`/gauth/pep/v1/enforce/batch`	`{ requests: EnforcementRequest[], mode: string }`	`BatchDecision` or `EnforcementError`	Batch enforcement
`POST`	`/gauth/pep/v1/policy`	`{ credential: CredentialReference }`	`EnforcementPolicy` or `EnforcementError`	Policy inspection
`GET`	`/gauth/pep/v1/health`	—	`{ status: "ok", pep_version: string, interface_version: "1.2" }`	Health and version discovery

8.3 Request/Response Format

- Content-Type: `application/json`

- Accept: `application/json`

- Character encoding: UTF-8

8.4 HTTP Status Codes

```
| Scenario | HTTP Status | Body |

|----------|-------------|------|

| Successful enforcement (PERMIT, DENY, or CONSTRAIN) | `200
OK` | `EnforcementDecision` |

| Invalid request | `400 Bad Request` | `EnforcementError`
with `error_code: "INVALID_REQUEST"` |

| Credential parse failure | `400 Bad Request` |
`EnforcementError` with `error_code: "CREDENTIAL_PARSE_ERROR"`
|

| Issuer unreachable (stateful mode) | `502 Bad Gateway` |
`EnforcementError` with `error_code: "ISSUER_UNREACHABLE"` |

| Evaluation timeout | `504 Gateway Timeout` |
`EnforcementError` with `error_code: "EVALUATION_TIMEOUT"` |

| Internal PEP error | `500 Internal Server Error` |
`EnforcementError` with `error_code: "PEP_INTERNAL_ERROR"` |
```

Important: All three enforcement decisions (PERMIT, DENY, CONSTRAIN) return HTTP 200. The enforcement decision is a *policy outcome*, not an HTTP error. HTTP 4xx/5xx codes are reserved for PEP operational errors. Clients MUST inspect the `decision` field in the response body to determine the policy outcome.

8.5 Request Headers

```
| Header | Required | Description |

|--------|----------|-------------|

| `Content-Type` | Yes | MUST be `application/json` |

| `X-Request-ID` | No | Trace correlation ID. If provided, the
PEP SHOULD echo it in the response and use it as the
`request_id` if none is in the body. |

| `Authorization` | Depends | PEP-to-PEP or service-to-PEP
authentication. Format is deployment-specific (e.g., `Bearer
<token>`, `GAuth-S2S <key>`). The PEP SHOULD require
authentication for all HTTP endpoints. |
```

8.6 Response Headers

| Header | Description |

|--------|-------------|

| `X-PEP-Version` | Implementation version of the PEP |

| `X-PEP-Interface-Version` | RFC 0117 interface version (e.g., `1.1`) |

| `X-PEP-Processing-Time-Ms` | Processing time in milliseconds (also in the body) |

9. EVALUATION SEMANTICS

9.1 Evaluation Pipeline

The PEP evaluates enforcement checks in a defined order. Checks are short-circuit: if any check produces `fail` with `error` severity, subsequent checks MAY be skipped (but SHOULD still be executed for complete audit records).

Evaluation order:

```
CHK-01: Credential Integrity       (signature, format, schema_version)
   |
CHK-02: Temporal & Status          (exp, nbf, mandate_status, agent binding)
   |
CHK-03: Governance Profile         (profile ceiling validation)
   |
CHK-04: Phase                      (plan/build/run phase check)
   |
CHK-05: Sector                     (allowed_sectors check)
   |
```

```
CHK-06: Region                      (allowed_regions check,
data residency)
   |
CHK-07: Path                        (allowed_paths /
denied_paths validation)
   |
CHK-08: Verb Permission             (core_verbs allowed check)
   |
CHK-09: Verb Constraints            (per-verb constraints:
path_patterns, commands, registries, file_size)
   |
CHK-10: Platform Permissions        (Layer 3 permissions —
implementation-defined)
   |
CHK-11: Transaction Type & Matrix   (allowed_transactions,
transaction_matrix cross-check)
   |
CHK-12: Decision Type               (allowed_decisions
precedence check)
   |
CHK-13: Budget                      (budget remaining vs.
estimated cost)
   |
CHK-14: Session Limits              (tool calls, duration,
lines per commit)
   |
CHK-15: Approval                    (approval mode + action-
specific approval requirements)
   |
CHK-16: Delegation Chain            (chain validity, depth,
scope narrowing)
```

9.2 Check Specifications

CHK-01: Credential Integrity

- Input: `credential` from the enforcement request
- PoA fields evaluated: JWT signature, `schema_version`, `scope_checksum`

Logic:

```
1. If credential.format == "jwt":
   a. Verify JWT signature against issuer JWKS (kid → public key)
   b. Verify schema_version matches a recognized version (RFC 0116 §13.3)
   c. Verify scope_checksum matches recomputed checksum (RFC 0116 §4.4)
2. If credential.format == "w3c_vc":
   a. Verify Data Integrity Proof (RFC 0116 §6.3)
   b. Verify credential schema conformance
3. If credential.format == "sd-jwt":
   a. Verify SD-JWT signature and disclosures
   b. Verify holder binding (KB-JWT)
4. If any verification fails → FAIL (CREDENTIAL_INVALID)
```

CHK-02: Temporal & Status Validity

- Input: `credential`, `agent`, `context.live_mandate_state` (if stateful)
- PoA fields evaluated: `exp`, `nbf`, `iat`, `aud`, `mandate_status`, `sub`

Logic:

```
```

```
1. Check exp > current_time → FAIL if expired
(CREDENTIAL_EXPIRED)

2. Check nbf <= current_time → FAIL if not yet valid

3. Check aud contains the validating service URI

4. Check agent.agent_id matches credential subject → FAIL if
mismatch (AGENT_MISMATCH)

5. If stateful mode AND live_mandate_state available:

   a. Check status == "active" → FAIL if revoked
(CREDENTIAL_REVOKED),

      expired (CREDENTIAL_EXPIRED), superseded
(CREDENTIAL_SUPERSEDED),

      or budget_exceeded (BUDGET_EXHAUSTED)
```

CHK-03: Governance Profile Ceiling

- Input: `action`, PoA `scope.governance_profile`
- PoA fields evaluated: `governance_profile` + profile ceiling table (RFC 0115)

Logic:

```
1. Look up the governance profile ceiling for the PoA's
governance_profile

   (profile ceiling tables are defined in RFC 0115)

2. For stateless ceiling attributes:

   a. If action requires auto-deploy AND profile ceiling
disallows → FAIL (PROFILE_CEILING_EXCEEDED)

   b. If action targets a deployment environment not in
profile ceiling → FAIL

   c. If action approval_mode is less strict than profile
minimum → FAIL

3. For stateful ceiling attributes (requires stateful mode):

   a. Check resource access permissions against ceiling
```

```
   b. Check audit requirements against ceiling

   c. Check delegation permission against ceiling
```

CHK-04: Phase

- Input: `action`, PoA `scope.phase`
- PoA fields evaluated: `phase`

Logic:

```
1. Map the action verb to its permitted phases:

   - plan: read-only actions, analysis, planning

   - build: file operations, dependency management, command execution

   - run: deployment, production operations

2. If the action verb is not permitted in the PoA's current phase → FAIL (PHASE_MISMATCH)
```

Phase-to-verb mapping is implementation-specific, but implementations SHOULD follow the convention that `plan` is the most restrictive (read-only), `build` permits write operations to the codebase, and `run` permits deployment and production operations.

CHK-05: Sector

- Input: `action.sector`, PoA `scope.allowed_sectors`
- PoA fields evaluated: `allowed_sectors` (array of sector identifiers, e.g., NAICS codes)

Logic:

```
1. If PoA does not define allowed_sectors (absent or empty) → SKIP (no sector restriction)

2. If PoA defines allowed_sectors AND action.sector is absent:

   → The behavior depends on the enforcement policy:

     a. Strict mode (RECOMMENDED for behoerde profiles): FAIL (SECTOR_MISMATCH)

        — sector-restricted PoAs require explicit sector context on every action

     b. Permissive mode: PASS with warning

        — the action is treated as implicitly within the PoA's sector scope

   Implementations MUST document which mode they use.

3. If action.sector is not in PoA allowed_sectors → FAIL (SECTOR_MISMATCH)

4. If action.sector is in PoA allowed_sectors → PASS
```

Precedence: Sector checks run before region checks. A PoA restricted to a specific sector rejects all actions in other sectors regardless of region or verb.

CHK-06: Region

- Input: `action.region`, PoA `scope.allowed_regions`
- PoA fields evaluated: `allowed_regions` (array of region identifiers, e.g., ISO 3166-1 alpha-2 country codes or supranational identifiers like `EU`)

Logic:

```
1. If PoA does not define allowed_regions (absent or empty) → SKIP (no region restriction)

2. If PoA defines allowed_regions AND action.region is absent:
```

```
   → Same strict/permissive behavior as CHK-05.

3. Resolve supranational membership:

   a. If action.region == "DE" and allowed_regions contains
"EU" → match (DE is an EU member)

   b. Supranational resolution follows ISO 3166-1 membership
tables

4. If action.region is not in PoA allowed_regions (after
supranational resolution) → FAIL (REGION_MISMATCH)

5. If action.region is in PoA allowed_regions → PASS
```

Data residency: Region enforcement is critical for public-administration governance profiles. PoAs with `allowed_regions: ["DE"]` MUST reject actions that would cause data processing outside Germany, even if the verb and sector are permitted.

CHK-07: Path Validation

- Input: `action.resource`, PoA `scope.allowed_paths`, `scope.denied_paths`
- PoA fields evaluated: `allowed_paths`, `denied_paths`

Logic:

```
1. If action.resource_type is not path-based (e.g., "shell",
"api") → SKIP

2. Normalize the resource path (resolve relative paths, remove
trailing slashes)

3. Check denied_paths first (deny takes precedence):

   a. If resource matches any denied_path glob pattern → FAIL
(PATH_DENIED)

4. Check allowed_paths:

   a. If allowed_paths is empty → PASS (no path restriction)

   b. If resource matches any allowed_path glob pattern → PASS
```

```
   c. If resource does not match any allowed_path → FAIL
(PATH_NOT_ALLOWED)
```

Precedence rule: `denied_paths` takes precedence over `allowed_paths`. A path that matches both is DENIED.

CHK-08: Verb Permission

- Input: `action.verb`, PoA `scope.core_verbs`
- PoA fields evaluated: `core_verbs`

Logic:

```
1. Look up action.verb in the PoA's core_verbs object

2. If verb is not present in core_verbs → FAIL
(VERB_NOT_ALLOWED)

3. If core_verbs[verb].allowed == false → FAIL
(VERB_NOT_ALLOWED)

4. If core_verbs[verb].allowed == true → PASS (verb is
permitted)
```

In stateless mode, the PEP uses `tool_permissions_hash` to verify the core_verbs have not been tampered with, but does not have access to the full core_verbs object. In this case, the PEP MUST use stateful mode for verb-level enforcement, or pre-load the permission set.

CHK-09: Verb Constraints

- Input: `action`, PoA `scope.core_verbs[verb].constraints`
- PoA fields evaluated: `core_verbs[verb].constraints`

Logic:

```
1. If core_verbs[verb].constraints is absent → PASS (no constraints)
2. For each applicable constraint:
   a. path_patterns:
      - If action.resource does not match any path_patterns glob → FAIL (CONSTRAINT_VIOLATED)
      - If matches → CONSTRAIN (path_restricted)
   b. allowed_commands:
      - If action command not in allowed_commands → FAIL (CONSTRAINT_VIOLATED)
      - If matches → CONSTRAIN (command_restricted)
   c. denied_commands:
      - If action command in denied_commands → FAIL (CONSTRAINT_VIOLATED)
   d. max_delegation_depth:
      - If action is delegation AND delegation depth >= max_delegation_depth → FAIL (CONSTRAINT_VIOLATED)
      - If within limit → CONSTRAIN (delegation_depth_limited)
   e. max_file_size_bytes:
      - If action content exceeds max_file_size_bytes → FAIL (CONSTRAINT_VIOLATED)
      - If within limit → CONSTRAIN (file_size_limited)
```

CHK-10: Platform Permissions (Layer 3)

- Input: `action`, PoA `scope.platform_permissions`
- PoA fields evaluated: `platform_permissions`

Logic:

```
1. Map the action to a platform permission category.

   The mapping from action verbs/resource types to permission
categories is

   implementation-defined. Common categories include
deployment, database,

   shell, packages, secrets, and external APIs.


2. Evaluate the specific permission:

   a. Check that the agent has the required permission for the
action

   b. Check any fine-grained constraints (e.g., target
environments, access modes)


3. If permission check fails → FAIL
(PLATFORM_PERMISSION_DENIED)
```

In stateless mode, the PEP uses `platform_permissions_hash` to verify integrity but does not have access to the full permission object. The PEP MUST use stateful mode for platform permission enforcement, or pre-load the permission set.

CHK-11: Transaction Type & Matrix

- Input: `action.transaction_type`, PoA `scope.allowed_transactions`, `scope.transaction_matrix`
- PoA fields evaluated: `allowed_transactions`, `transaction_matrix`

Logic:

```
1. If PoA does not define allowed_transactions AND
transaction_matrix → SKIP
```

```
2. If allowed_transactions is defined:

   a. If action.transaction_type is absent → FAIL
(TRANSACTION_NOT_ALLOWED)

   b. If action.transaction_type is not in
allowed_transactions → FAIL (TRANSACTION_NOT_ALLOWED)

3. If transaction_matrix is defined:

   a. Look up the matrix entry for (action.verb,
action.transaction_type)

   b. If no matrix entry exists for this combination → FAIL
(TRANSACTION_NOT_ALLOWED)

   c. If matrix entry has constraints → CONSTRAIN
(transaction_restricted)

      with the matrix-defined constraints (e.g., amount
limits, time windows)

4. PASS
```

Transaction matrix semantics: The transaction matrix is a cross-product of (verb × transaction_type) that defines fine-grained permissions beyond simple verb allow/deny. The matrix entry overrides the verb-level permission for that specific transaction type.

Precedence: Transaction matrix entries take precedence over simple `allowed_transactions` membership. If a transaction_type is in `allowed_transactions` but the matrix entry for (verb, transaction_type) denies it, the matrix entry wins.

CHK-12: Decision Type

- Input: `action.decision_type`, PoA `scope.allowed_decisions`
- PoA fields evaluated: `allowed_decisions` (array of permitted decision classifications)

Logic:

```
1. If PoA does not define allowed_decisions → SKIP (no
decision-type restriction)
```

```
2. If action.decision_type is absent AND PoA defines
allowed_decisions → FAIL (DECISION_NOT_ALLOWED)

   (decision-restricted PoAs require decision context)

3. If action.decision_type is not in allowed_decisions → FAIL
(DECISION_NOT_ALLOWED)

4. If action.decision_type is in allowed_decisions → PASS
```

Decision type semantics: Decision types classify the nature of the outcome an agent action produces — e.g., `informational` (read-only analysis), `advisory` (non-binding recommendation), `binding` (creates obligations), `financial` (involves monetary transactions). PoAs can restrict which decision types an agent may produce, independent of which verbs it may execute.

Precedence with approval: If a decision type requires multi-approver approval (per governance profile or per-decision configuration), the decision type check passes but CHK-15 (Approval) enforces the multi-approver requirement. The two checks are complementary.

CHK-13: Budget

- Input: `action.parameters.amount_cents`, PoA budget claims or live mandate state
- PoA fields evaluated: `budget.remaining_cents`, `budget.total_cents`

Logic:

```
1. Determine estimated cost:

   a. Use action.parameters.amount_cents if provided

   b. Otherwise, use core_verbs[verb].cost_cents_base

   c. If neither available → SKIP (no budget enforcement for
this action)


2. Determine remaining budget:
```

```
   a. Stateful mode: use
context.live_mandate_state.budget_remaining_cents

   b. Stateless mode: use credential's budget.remaining_cents

      (NOTE: stateless budget may be stale; see Section 10.5)


3. If remaining_budget == 0 → FAIL (BUDGET_EXHAUSTED)

4. If estimated_cost > remaining_budget → FAIL
(BUDGET_EXCEEDED)

5. If remaining_budget is below a low-budget threshold →
CONSTRAIN (budget_warning)

   Implementations SHOULD define their own low-budget
threshold

   (e.g., remaining < 120% of estimated cost, or remaining <
10% of total).

   The threshold is implementation-specific and not normative.
```

CHK-14: Session Limits

- Input: `context.session_state`, PoA `requirements.session_limits`
- PoA fields evaluated: `session_limits.max_tool_calls`, `session_limits.max_lines_per_commit`, `requirements.ttl_seconds`

Logic:

```
1. If context.session_state is absent → SKIP (no session
tracking)

2. Check tool calls:

   a. If max_tool_calls is defined AND
session_state.tool_calls_used >= max_tool_calls

      → FAIL (SESSION_LIMIT_EXCEEDED)

3. Check session duration:
```

```
   a. If ttl_seconds is defined AND (current_time -
session_started_at) > ttl_seconds

      → FAIL (SESSION_LIMIT_EXCEEDED)

4. Check lines per commit:

   a. If max_lines_per_commit is defined AND action involves
file write:

      Enforce constraint → CONSTRAIN (lines_limited)
```

CHK-15: Approval

- Input: `action`, PoA `requirements.approval_mode`
- PoA fields evaluated: `approval_mode`, `core_verbs[verb].requiresApproval`

Logic:

```
1. Check PoA approval mode:

   a. "autonomous": no approval required for any action → PASS

   b. "supervised": approval required for actions in the
requiresApprovalFor list

   c. "four-eyes": approval required from multiple approvers
(min 2)


2. If approval is required for this action:

   a. Check if approval has been granted (via approval_token
or approval record)

   b. If not granted → FAIL (APPROVAL_REQUIRED)

   c. For "four-eyes": verify that the required number of
distinct approvers have signed


3. If the specific verb has requiresApproval == true in
core_verbs:
```

```
   a. Apply same logic regardless of global approval_mode
```

CHK-16: Delegation Chain

- Input: `credential.delegation_chain`, `action`
- PoA fields evaluated: `delegation_chain` (RFC 0116 §4.3)

Logic:

See Section 11 for full delegation chain enforcement semantics.

9.3 Check Applicability

Not all checks apply to every action. The PEP MUST evaluate applicability based on the action type and PoA content:

```
| Check | Applies When |
|-------|-------------|
| CHK-01 (Credential) | Always |
| CHK-02 (Temporal/Status) | Always |
| CHK-03 (Profile Ceiling) | Always |
| CHK-04 (Phase) | Always |
| CHK-05 (Sector) | PoA defines `allowed_sectors` |
| CHK-06 (Region) | PoA defines `allowed_regions` |
| CHK-07 (Path) | `resource_type` is path-based (file,
directory) |
| CHK-08 (Verb) | Always |
| CHK-09 (Constraints) | `core_verbs[verb].constraints` is
present |
| CHK-10 (Platform) | Action maps to a platform permission |
| CHK-11 (Transaction) | PoA defines `allowed_transactions` or
`transaction_matrix` |
```

```
| CHK-12 (Decision) | PoA defines `allowed_decisions` |

| CHK-13 (Budget) | PoA has budget claims AND action has cost
|

| CHK-14 (Session) | PoA has session_limits AND session_state
is provided |

| CHK-15 (Approval) | PoA approval_mode != "autonomous" OR
verb has requiresApproval |

| CHK-16 (Delegation) | `delegation_chain` is non-empty |
```

When a check does not apply, the PEP returns `result: "skip"` for that check.

10. STATELESS VS. STATEFUL ENFORCEMENT

10.1 Enforcement Modes

The PEP operates in one of two modes, mirroring RFC 0116 §10.2 validation modes:

- **Stateless**: PoA snapshot from the JWT/VC only | < 5ms | Read operations, high-frequency actions, edge enforcement |
- **Stateful**: PoA snapshot + live mandate state from issuer | 50-100ms | Write operations, budget-sensitive actions, high-value decisions |

10.2 Stateless Enforcement

In stateless mode, the PEP evaluates the action against the frozen PoA snapshot in the presented credential. No network calls are made to the issuer.

Available checks in stateless mode:

```
| Check | Available | Notes |

|-------|-----------|-------|

| CHK-01 (Credential) | Full | Signature verification against
cached JWKS |

| CHK-02 (Temporal) | Partial | `exp`/`nbf`/`aud` from JWT; no
live status check |

| CHK-03 (Profile) | Stateless attributes only | Stateless
ceiling attributes (session duration, tool calls, lines per
commit, approval mode) |
```

```
| CHK-04 (Phase) | Full | Phase is in the PoA |

| CHK-05 (Sector) | Full | Allowed sectors are in the PoA |

| CHK-06 (Region) | Full | Allowed regions are in the PoA |

| CHK-07 (Path) | Full | Paths are in the PoA |

| CHK-08 (Verb) | Via hash only | `tool_permissions_hash`
integrity check; no full verb lookup |

| CHK-09 (Constraints) | No | Requires full core_verbs object
(not in JWT size budget) |

| CHK-10 (Platform) | Via hash only |
`platform_permissions_hash` integrity check; no full lookup |

| CHK-11 (Transaction) | Full | Allowed transactions and
matrix are in the PoA |

| CHK-12 (Decision) | Full | Allowed decisions are in the PoA
|

| CHK-13 (Budget) | Stale | Uses JWT budget claims, which may
lag behind actual spend |

| CHK-14 (Session) | If session_state provided | Session
tracking is caller-side |

| CHK-15 (Approval) | No | Requires approval record lookup |

| CHK-16 (Delegation) | Partial | Chain structure is in the
PoA; scope validation limited |
```

Limitations: Stateless enforcement cannot verify live mandate status (revocation, budget updates), cannot evaluate full permission objects (only hashes), and cannot check approval records.

10.3 Stateful Enforcement

In stateful mode, the PEP supplements the PoA snapshot with live data from the issuer. This requires a network call to the issuer's mandate store (via introspection endpoint or direct lookup).

Additional data available in stateful mode:

```
| Data | Source | Purpose |

|------|--------|---------|
```

| Mandate status | Issuer mandate store | Live revocation/expiry/superseded check |

| Budget remaining | Issuer mandate store | Accurate budget enforcement |

| Full tool_permissions | Issuer mandate store | Verb-level constraint enforcement |

| Full platform_permissions | Issuer mandate store | Platform permission enforcement |

| Approval records | Issuer approval store | Approval verification |

| Agent capability snapshot | Issuer mandate store | Capability validation |

When stateful mode is REQUIRED:

| Scenario | Reason |

|----------|--------|

| Write operations (file create/modify/delete) | Budget impact, audit integrity |

| Deployment operations | High-value, irreversible |

| Shell command execution | Security-sensitive |

| Agent delegation | Chain validation requires full scope |

| Budget-impacting operations | Stale budget in JWT may allow over-spend |

| Approval-gated operations | Approval records not in JWT |

10.4 Mode Selection Logic

The PEP SHOULD use the following logic to select enforcement mode:

```
IF governance_profile requires stateful_only (e.g., public-administration profiles):
    mode = stateful
```

```
ELSE IF action is read-only AND no budget impact AND
approval_mode == "autonomous":
    mode = stateless


ELSE:
    mode = stateful
```

Implementations MAY enforce in stateful mode for all operations. Stateless mode is an optimization for latency-sensitive paths, not a relaxation of security.

Governance profiles MAY declare a `stateful_only` attribute in their ceiling table (RFC 0115) to force all enforcement through stateful mode. This is RECOMMENDED for high-assurance profiles.

10.5 Stale Data Handling

In stateless mode, the PoA snapshot may contain stale data (budget, session counters). The PEP MUST:

1. Treat stateless budget as an upper bound — the actual remaining budget may be lower

2. Accept that a stateless PERMIT may be overridden by a subsequent stateful check

3. Log when stateless enforcement was used, so audit trails can differentiate stateless vs. stateful decisions

11. DELEGATION CHAIN ENFORCEMENT

11.1 Delegation Model

RFC 0116 §4.3 defines a `delegation_chain` array in the PoA credential. Each entry in the chain represents a delegation from one entity to another, with an optional scope restriction.

11.2 Delegation Checks

When `delegation_chain` is non-empty, the PEP MUST perform the following checks:

```
1. Chain Integrity:

   a. Verify the chain is ordered from original principal to
current holder

   b. Verify each entry has required fields (delegator,
delegate, scope_restriction, delegated_at)

   c. Verify the last delegate in the chain matches the
presenting agent


2. Depth Check:

   a. Count the chain length

   b. If the PoA's core_verbs contain a max_delegation_depth
constraint:

      - If chain length > max_delegation_depth → FAIL
(DELEGATION_DEPTH_EXCEEDED)

   c. If the governance profile has a delegation depth
ceiling:

      - If chain length > profile ceiling → FAIL
(DELEGATION_DEPTH_EXCEEDED)


3. Scope Narrowing:

   a. For each entry in the chain (from first to last):

      - Verify scope_restriction is a subset of the
delegator's effective scope

      - The delegate's effective scope is the intersection of
the delegator's scope and the scope_restriction

   b. If any scope_restriction exceeds the delegator's scope →
FAIL (DELEGATION_SCOPE_EXCEEDED)

   c. The final effective scope (after all narrowings) is the
scope used for all subsequent PEP checks


4. Temporal Validity:

   a. For each entry: delegated_at MUST be <= current_time
```

```
   b. For each entry: delegated_at MUST be within the parent
credential's validity period


5. max_depth_remaining:

   a. For each entry: max_depth_remaining MUST be >= 0

   b. Each subsequent entry's max_depth_remaining MUST be <=
(previous entry's max_depth_remaining - 1)
```

11.3 Effective Scope After Delegation

The delegation chain check (CHK-16) runs last in the evaluation pipeline. When the delegation chain is valid, the effective scope is the narrowed result:

```
effective_scope = original_poa_scope

FOR each entry IN delegation_chain:

    effective_scope = intersect(effective_scope,
entry.scope_restriction)
```

Two-pass evaluation: Implementations MUST perform a two-pass evaluation when the delegation chain narrows the effective scope:

1. **Pass 1**: Run CHK-01 through CHK-15 against the original PoA scope (confirms the PoA itself authorizes the action class).

2. **Pass 2**: Run CHK-16 to validate the delegation chain and compute `effective_scope`.

3. **Re-evaluate** CHK-05 (Sector), CHK-06 (Region), CHK-07 (Path), CHK-08 (Verb), CHK-09 (Constraints), CHK-10 (Platform), CHK-11 (Transaction), and CHK-12 (Decision) against the `effective_scope` if it differs from the original scope.

This two-pass approach ensures that a delegated agent cannot exercise permissions broader than what was delegated, even if the underlying PoA authorizes them.

Security note: A single-pass implementation that only validates the delegation chain structure without re-evaluating the narrowed scope is NOT conformant. Such an implementation would allow a delegated agent to exercise the full scope of the original PoA, defeating the purpose of scope narrowing.

12. INTERFACE VERSIONING

12.1 Version Identification

The PEP interface version follows the format `{major}.{minor}` (e.g., `1.1`). This version is distinct from the implementation version — a single implementation may support multiple interface versions.

The current interface version defined by this document is **1.1**.

12.2 Version Discovery

PEP implementations MUST expose their supported interface version via:

1. The `audit.pep_interface_version` field in every `EnforcementDecision`
2. The `X-PEP-Interface-Version` response header (for HTTP-binding deployments)
3. The `/gauth/pep/v1/health` endpoint (for HTTP-binding deployments)

12.3 Compatibility Rules

```
| Change Type | Version Impact | Backward Compatible |
|-------------|---------------|-------------------|
| Adding a new optional field to EnforcementRequest | Minor bump | Yes — older PEPs ignore unknown fields |
| Adding a new check to the evaluation pipeline | Minor bump | Yes — older PEPs skip unknown checks |
| Adding a new violation code | Minor bump | Yes — clients treat unknown codes as generic violations |
```

| Changing a required field or removing a field | Major bump | No — breaks existing clients |

| Changing evaluation order or precedence | Major bump | No — changes decision outcomes |

| Adding a new decision type (beyond PERMIT/DENY/CONSTRAIN) | Major bump | No — clients may not handle it |

12.4 Negotiation

For HTTP-binding deployments, clients MAY indicate their preferred interface version via the `Accept` header:

```
Accept: application/json; gauth-pep-version=1.1
```

If the PEP does not support the requested version, it SHOULD return `406 Not Acceptable` with a list of supported versions.

13. DEPLOYMENT PATTERNS (Non-Normative)

This section describes four deployment patterns for integrating the PEP into a service architecture. These patterns are non-normative guidance — implementations MAY use any pattern that satisfies the PEP interface contract (Sections 4-8) and evaluation semantics (Section 9).

13.1 Pattern A: Middleware / Interceptor

The PEP is implemented as a library that plugs into the service's request pipeline (e.g., Express middleware, gRPC interceptor, Spring Security filter).

```
┌──────────────────────────────────────────────┐
│  Service Process                              │
│                                               │
│  Request → [Auth MW] → [PEP MW] → Handler    │
│                          │                    │
│                          └── enforceAction()│
```

```
│                                (in-process)    │
└────────────────────────────────────────────────┘
```

Characteristics:

```
| Aspect | Value |
|--------|-------|
| Latency | Lowest (no network hop, <1ms overhead) |
| Coupling | Tight — PEP library is a dependency of the service |
| Language dependency | PEP must be implemented in the service's language |
| Operational complexity | Lowest — PEP upgrades require service redeployment |
| Stateful mode | PEP makes outbound call to issuer for live mandate state |
```

When to use: Single-language environments where the service team owns both the service and PEP enforcement. Best for high-throughput, latency-sensitive paths.

13.2 Pattern B: Sidecar / Proxy

The PEP runs as a standalone process alongside the service, intercepting requests before they reach the service handler. Similar to an Envoy ext_authz filter or a dedicated GAuth PEP proxy.

```
┌──────────────────────────────────────────────────────────┐
│  Pod / Container Group                                    │
│                                                          │
│  ┌──────────────┐         ┌──────────────────────┐      │
│  │ PEP Proxy    │         │ Service              │      │
│  │              │──allow──│                      │      │
│  │ :8443        │         │ :8080                │      │
```

```
│   │ enforceAction│          │ (no PEP logic)         │ │
│   └──────────────┘          └────────────────────────┘ │
└────────────────────────────────────────────────────────┘
```

Characteristics:

```
| Aspect | Value |
|--------|-------|
| Latency | Low (~1-5ms network hop, localhost) |
| Coupling | Loose — PEP is a separate process |
| Language dependency | None — PEP can be any language |
| Operational complexity | Medium — separate process to manage |
| Stateful mode | PEP makes outbound call to issuer for live mandate state |
```

When to use: Multi-language environments, microservice architectures, or when the service team does not want PEP logic in their codebase. Works well with service mesh patterns (Envoy, Istio).

The HTTP binding (Section 8) defines the API surface for this pattern.

13.3 Pattern C: SDK / Client Library

The PEP is provided as a client library that services call explicitly at decision points. Unlike the middleware pattern (A), the service chooses when and where to call the PEP.

```
┌──────────────────────────────────────────────────────┐
│  Service Process                                      │
│                                                      │
│  Handler:                                            │
│    ...                                               │
│    decision = pepClient.enforceAction(request)       │
```

```
│    if (decision.decision == "DENY") return 403   │
│    ...proceed with action...                      │
│                                                   │
└───────────────────────────────────────────────────┘
```

Characteristics:

Aspect	Value
Latency	Same as Pattern A (in-process) or Pattern B (if SDK wraps a remote call)
Coupling	Medium — SDK is a dependency, but not in the request pipeline
Language dependency	SDK must be available for the service's language
Operational complexity	Low — SDK updates via package management
Stateful mode	SDK makes outbound call to issuer for live mandate state

When to use: When enforcement points are not at request boundaries (e.g., mid-workflow decisions, background jobs, multi-step transactions). Maximum flexibility — the developer controls when PEP runs.

Tradeoff: The developer must remember to call the PEP. Missing a call-site means an unenforced action. Patterns A and B eliminate this risk by intercepting all requests.

13.4 Pattern D: MCP Compliance Proxy

The PEP is combined with additional compliance logic in a proxy that sits before MCP tool servers. The proxy performs the normative PEP checks (this specification) and may add non-normative enrichment (threat scanning, architecture compliance, prompt injection detection) in a separate layer.

```
┌──────────────────────────────────────────────────────────────────┐
│  MCP Call Flow                                                   │
│                                                                  │
│  Caller → [PEP Checks (RFC 0117)] → [Compliance Enrichment]    │
│           │                          │                           │
│           │ enforceAction()          │ (non-normative,           │
│           │ (normative)              │  platform-specific)       │
│           │                          │                           │
│           └──────────────────────────┴──→ Target MCP Server     │
│                                                                  │
│  Response ← [Outbound Compliance] ← Target MCP Server          │
└──────────────────────────────────────────────────────────────────┘
```

Characteristics:

```
| Aspect | Value |
|--------|-------|
| Latency | Higher (~50ms PEP + variable for enrichment) |
| Coupling | Loose — proxy is a separate service |
| Language dependency | None — proxy can be any language |
| Operational complexity | Highest — separate service with additional compliance logic |
| Stateful mode | PEP makes outbound call to issuer; enrichment layer adds additional analysis |
```

When to use: Agent-based systems where MCP tool calls are the primary action surface. Combines authorization (PEP) with compliance in a single interception point.

Critical distinction: The normative PEP checks (this specification) are deterministic and protocol-level. Any compliance enrichment layer is non-normative and platform-specific. Implementations MUST clearly separate PEP decisions from enrichment verdicts in their output.

14. CONFORMANCE REQUIREMENTS

14.1 GAuth-Conformant PEP

A GAuth-conformant PEP MUST:

1. Accept `EnforcementRequest` conforming to Section 4.1

2. Return `EnforcementDecision` conforming to Section 5.1

3. Return `EnforcementError` conforming to Section 6.1 when evaluation cannot complete

4. Implement `enforceAction()` as defined in Section 7.1

5. Execute the evaluation pipeline (Section 9.1) in the specified order

6. Evaluate all applicable checks for each enforcement request

7. Return `DENY` when any check produces `fail` with `error` severity

8. Return `CONSTRAIN` when checks pass but constraints apply

9. Return `PERMIT` only when all applicable checks pass without constraints

10. Default to `DENY` when evaluation cannot complete (fail-closed)

11. Include a complete `checks` array in every decision, listing all evaluated checks

12. Include an `audit` record in every decision with processing time, credential reference, and `pep_interface_version`

13. Support stateless enforcement mode (Section 10.2)

14. Support stateful enforcement mode (Section 10.3) for write operations

15. Validate PoA credentials per RFC 0116 §10.2 before evaluation

16. Enforce `denied_paths` with higher precedence than `allowed_paths` (Section 9.2, CHK-07)

17. Perform two-pass evaluation for delegation chains that narrow the effective scope (Section 11.3)

A GAuth-conformant PEP SHOULD:

1. Implement `batchEnforce()` as defined in Section 7.2

2. Implement `getEnforcementPolicy()` as defined in Section 7.3

3. Execute all 16 checks even after a fail (for complete audit records)

4. Cache JWKS with a TTL of no more than 1 hour for stateless credential validation

5. Log enforcement decisions per RFC 0116 §12.4

6. Support delegation chain enforcement (Section 11) when `delegation_chain` is present in the PoA

7. Report violation codes from the registry (Section 5.3) for machine-readable error handling

8. Include `enforcement_mode` in the decision to distinguish stateless from stateful evaluation

14.2 GAuth-Conformant PEP Client

A service that delegates enforcement to an external PEP (Pattern B or D) MUST:

1. Forward all required `EnforcementRequest` fields to the PEP

2. Respect the `EnforcementDecision` — reject actions when decision is `DENY`

3. Apply `enforced_constraints` when decision is `CONSTRAIN`

4. Treat `EnforcementError` responses as equivalent to `DENY` (fail-closed)

5. Forward the `audit` record to the audit logging system

14.3 Interaction with RFC 0116 Conformance

GiFo-RFC 0117 conformance is independent of, but complementary to, GiFo-RFC 0116 conformance:

- A GAuth-conformant verifier (RFC 0116 §13.2) validates credentials but does not enforce actions

- A GAuth-conformant PEP (RFC 0117 §14.1) enforces actions using validated credentials

- A complete GAuth enforcement stack implements both: credential validation (RFC 0116) + action enforcement (RFC 0117)

15. SECURITY CONSIDERATIONS

15.1 PEP Bypass

The PEP is only effective if all actions pass through it. Implementations MUST ensure:

| Bypass Vector | Mitigation |

|---|---|

| Direct access to target resource (bypassing PEP) | Network-level enforcement: target resources MUST only accept requests from the PEP or PEP-authorized paths |

| Service-to-service calls that skip PEP | All inter-service calls carrying PoA credentials MUST pass through a PEP |

| Client-side PEP evaluation | PEP MUST run server-side. Client-side enforcement is advisory only |

15.2 Fail-Open vs. Fail-Closed

The PEP MUST operate in fail-closed mode:

| Failure Scenario | PEP Behavior |

|---|---|

| Credential validation error | DENY |

| Issuer unreachable (stateful mode) | DENY |

| Timeout during evaluation | DENY |

| Unknown action verb | DENY |

| Unknown governance profile | DENY |

| Internal PEP error | DENY |

| Missing required field in EnforcementRequest | Return error response (INVALID_REQUEST) |

There is no fail-open mode. Implementations MUST NOT provide a configuration option to enable fail-open behavior.

15.3 Decision Caching

Implementations MAY cache enforcement decisions for performance, subject to these constraints:

| Constraint | Requirement |

|---|---|

| Cache key | MUST include: credential JTI, action verb, resource, timestamp granularity |

```
| Cache TTL | MUST NOT exceed 60 seconds for stateless
decisions |

| No caching for stateful | Stateful decisions MUST NOT be
cached (budget and status may change between requests) |

| Cache invalidation | MUST invalidate on: key rotation,
credential revocation, mandate status change |
```

Risk: Caching a PERMIT decision may allow an action that was subsequently DENIED (e.g., due to budget exhaustion or mandate revocation between cache write and cache read). Short TTLs and cache invalidation events mitigate this risk.

15.4 Audit Trail Integrity

The PEP audit record (Section 5.1, `audit` field) MUST:

1. Be immutable once written — no post-hoc modification of enforcement decisions
2. Include the enforcement mode (stateless/stateful) to distinguish decision quality
3. Include processing time for performance monitoring and SLA enforcement
4. Reference the credential JTI (not the full credential) for privacy
5. Be stored in a tamper-evident log if the deployment requires regulatory compliance

For deployments under public-administration governance profiles, audit records MUST comply with applicable public-sector record-keeping regulations (which may exceed the minimums in RFC 0116 §12.4).

15.5 Timing Attacks

The PEP evaluation pipeline SHOULD execute in approximately constant time for a given set of applicable checks in stateless mode, regardless of whether the decision is PERMIT or DENY. This prevents timing-based inference of the PoA's contents.

In stateful mode, variable latency from network calls is unavoidable. Implementations SHOULD add random jitter to stateful response times to prevent timing-based inference of enforcement mode or issuer response patterns.

Constant-time enforcement is primarily a concern for stateless mode. Stateful mode already introduces variable latency; the jitter recommendation mitigates residual timing signals.

15.6 Scope Confusion

The PEP MUST correctly handle scope boundaries to prevent scope confusion attacks:

```
| Attack | Mitigation |

|--------|-----------|

| Agent presents PoA for a different project | CHK-02 verifies
agent_id matches PoA subject |

| Agent modifies path after PEP check | PEP decision includes
the checked resource; the target resource MUST verify the
resource matches |

| Agent replays a PERMIT decision for a different action |
Each decision is bound to a specific request_id, verb, and
resource |

| Delegation chain manufactures broader scope | CHK-16
enforces scope narrowing at each delegation step; two-pass re-
evaluation is MUST (§11.3) |
```

15.7 Resource Exhaustion

Implementations MUST protect the PEP against resource exhaustion:

```
| Vector | Mitigation |

|--------|-----------|

| High-frequency enforcement requests | Rate limiting (per-
agent, per-service) |

| Large enforcement request payloads | Request size limits
(recommended: 64 KB max) |

| Large delegation chains | Maximum chain length (recommended:
10 entries) |

| Batch enforcement with many requests | Maximum batch size
(recommended: 50 requests) |
```

Appendix A: Reference Implementation Pseudocode

The following pseudocode illustrates a conformant `enforceAction()` implementation. It is language-agnostic and intended for reference only.

```

function enforceAction(request: EnforcementRequest) →
EnforcementDecision:

    start_time = now()

    checks = []

    violations = []

    constraints = []

    mode = selectMode(request)


    // Resolve PoA data

    poa = null

    if request.credential.poa_snapshot:

        poa = request.credential.poa_snapshot

    elif request.credential.token:

        poa = validateAndParseCredential(request.credential)


    if poa == null:

        return denyDecision(request,
[violation("CREDENTIAL_INVALID", "CHK-01")])


    // If stateful, fetch live mandate state

    live_state = null

    if mode == "stateful" AND request.credential.mandate_id:

        live_state =
fetchMandateState(request.credential.mandate_id)

        if live_state == null:

            return denyDecision(request,
[violation("CREDENTIAL_INVALID", "CHK-02")])
```

```
    // Execute evaluation pipeline (all 16 checks in order per
§9.1)

    effective_scope = poa.scope


    pipeline = [

        ("CHK-01", "Credential Integrity",
checkCredentialIntegrity,  poa, request),

        ("CHK-02", "Temporal & Status",
checkTemporalStatus,      poa, request, live_state),

        ("CHK-03", "Governance Profile",
checkProfileCeiling,      poa.scope, request),

        ("CHK-04", "Phase",                     checkPhase,
poa.scope, request),

        ("CHK-05", "Sector",                    checkSector,
poa.scope, request),

        ("CHK-06", "Region",                    checkRegion,
poa.scope, request),

        ("CHK-07", "Path Validation",           checkPath,
poa.scope, request),

        ("CHK-08", "Verb Permission",           checkVerb,
poa.scope, request, live_state),

        ("CHK-09", "Verb Constraints",
checkVerbConstraints,     poa.scope, request, live_state),

        ("CHK-10", "Platform Permissions",
checkPlatformPermissions, poa.scope, request, live_state),

        ("CHK-11", "Transaction Type &
Matrix",checkTransaction,         poa.scope, request),

        ("CHK-12", "Decision Type",
checkDecisionType,        poa.scope, request),

        ("CHK-13", "Budget",                    checkBudget,
poa.scope, request, live_state),

        ("CHK-14", "Session Limits",
checkSessionLimits,       poa.scope, request),

        ("CHK-15", "Approval",                  checkApproval,
poa.scope, request, live_state),
```

```
    ]


    for (check_id, check_name, check_fn, ...args) in pipeline:

        result = check_fn(...args)

        checks.append(CheckResult(check_id, check_name,
result.result, result.detail))


        if result.result == "fail" AND
result.violation.severity == "error":

            violations.append(result.violation)

        elif result.result == "fail" AND
result.violation.severity == "warning":

            // Warning: log but do not add to blocking
violations

            violations.append(result.violation)

        elif result.result == "constrain":

            constraints.append(result.constraint)


    // CHK-16: Delegation Chain (last check)

    if poa.delegation_chain AND length(poa.delegation_chain) >
0:

        dc_result =
enforceDelegationChain(poa.delegation_chain, poa.scope,
request.agent)

        checks.append(CheckResult("CHK-16", "Delegation
Chain", dc_result.result, dc_result.detail))

        if dc_result.result == "fail":

            violations.append(dc_result.violation)

        else:

            effective_scope = dc_result.effective_scope

            // Two-pass: re-evaluate scope-dependent checks
against narrowed scope

            if effective_scope != poa.scope:
```

```
                recheck_ids = ["CHK-05", "CHK-06", "CHK-07",
"CHK-08", "CHK-09",
                               "CHK-10", "CHK-11", "CHK-12"]
                for recheck in recheck_ids:
                    re_result = reEvaluateCheck(recheck,
effective_scope, request, live_state)
                    checks.append(CheckResult(recheck + "-
DELEGATED", ..., re_result))
                    if re_result.result == "fail" AND
re_result.violation.severity == "error":
                        violations.append(re_result.violation)
    else:
        checks.append(CheckResult("CHK-16", "Delegation
Chain", "skip", "No delegation chain"))

    // Determine decision
    error_violations = filter(violations, severity == "error")
    decision = "PERMIT"
    if length(error_violations) > 0:
        decision = "DENY"
    elif length(constraints) > 0:
        decision = "CONSTRAIN"

    return EnforcementDecision(
        request_id = request.request_id,
        decision = decision,
        timestamp = now(),
        enforcement_mode = mode,
        checks = checks,
        enforced_constraints = constraints,
        violations = violations,
        audit = AuditRecord(
```

```
            processing_time_ms = elapsed(start_time),
            pep_version = PEP_VERSION,
            pep_interface_version = "1.2",
            credential_jti = poa.jti,
            mandate_id = request.credential.mandate_id,
            agent_id = request.agent.agent_id,
            action_verb = request.action.verb,
            action_resource = request.action.resource,
            checks_performed = length(checks),
            checks_passed = count(checks, result == "pass"),
            checks_failed = count(checks, result == "fail")
        )
    )
```

Appendix B: Deployment Pattern Tradeoff Matrix

Dimension	Pattern A (Middleware)	Pattern B (Sidecar)	Pattern C (SDK)	Pattern D (MCP Proxy)
Latency	<1ms	1-5ms	<1ms (in-process) / 1-5ms (remote)	50ms+ (PEP) + variable (enrichment)
Language coupling	Same language required	Language-agnostic	Same language required	Language-agnostic
Deployment coupling	Co-deployed with service	Co-located but separate process	Co-deployed with service	Separate service
Bypass risk	Low (middleware intercepts all)	Low (proxy intercepts all)	Medium (developer must call)	Low (proxy intercepts all)
Operational overhead	Low	Medium	Low	High

```
| **Multi-language support** | Per language | Once (any
language) | Per language | Once (any language) |

| **Enrichment capability** | None (PEP only) | None (PEP
only) | None (PEP only) | Full (PEP + compliance) |

| **Best for** | Single-language, high-throughput |
Microservices, service mesh | Workflow engines, background
jobs | Agent-based systems, MCP ecosystems |
```

Appendix C: PEP Claims Registry

The PEP Interface Contract introduces the following schema URIs:

```
| URI | Description |

|-----|-------------|

| `https://gimelfoundation.com/schemas/pep/v1.2/enforcement-
request.json` | EnforcementRequest schema (Section 4.1) |

| `https://gimelfoundation.com/schemas/pep/v1.2/enforcement-
decision.json` | EnforcementDecision schema (Section 5.1) |

| `https://gimelfoundation.com/schemas/pep/v1.2/enforcement-
error.json` | EnforcementError schema (Section 6.1) |
```

These schemas are published alongside the RFC 0116 PoA schemas. Implementations SHOULD validate requests and responses against these schemas.

Appendix D: Example Verb Namespaces

The following are non-normative examples of verb namespaces. Implementations define their own verb vocabularies within the `urn:gauth:verb:{domain}:{category}:{action}` format.

Example: Code Agent Domain

```
| Verb | URN |

|------|-----|

| Create file | `urn:gauth:verb:code:file:create` |

| Modify file | `urn:gauth:verb:code:file:modify` |

| Delete file | `urn:gauth:verb:code:file:delete` |
```

| Add dependency | `urn:gauth:verb:code:dependency:add` |

| Run command | `urn:gauth:verb:code:command:run` |

| Delegate to agent | `urn:gauth:verb:code:agent:delegate` |

Example: Financial Domain

| Verb | URN |

|------|-----|

| Create transaction | `urn:gauth:verb:finance:transaction:create` |

| Approve payment | `urn:gauth:verb:finance:payment:approve` |

| Query balance | `urn:gauth:verb:finance:balance:read` |

Example: Healthcare Domain

| Verb | URN |

|------|-----|

| Read patient record | `urn:gauth:verb:health:record:read` |

| Update diagnosis | `urn:gauth:verb:health:diagnosis:update` |

| Request referral | `urn:gauth:verb:health:referral:create` |

Appendix E: Open Questions

1. **PEP discovery mechanism**: Should there be a standardized endpoint for discovering PEP capabilities (supported checks, enforcement mode, version)? Analogous to RFC 0116 §13.4 GAuth Configuration Discovery. | Enables clients to discover PEP capabilities before submitting enforcement requests. The `/gauth/pep/v1/health` endpoint (§8.2) provides basic version discovery; a richer capability endpoint may be warranted. |
2. **Enforcement decision portability**: Should PEP decisions be signed/sealed so that downstream services can verify the PEP's identity and decision integrity without re-running enforcement? | Relevant for Pattern B/D where the PEP is separate from the target service |

3. **Custom check extensibility**: Should the evaluation pipeline support custom checks beyond the 16 defined here? If so, where in the pipeline do custom checks execute, and how are they registered? | Enables platform-specific enforcement without forking the specification |
4. **Real-time budget synchronization**: For high-concurrency environments, how should the PEP handle concurrent budget decrements that could cause race conditions? Should there be a reservation/pre-commit pattern? | Critical for multi-agent scenarios where budget is shared |
5. **Cross-PEP audit correlation**: When a single logical operation traverses multiple PEPs (e.g., delegation chain), how should audit records be correlated across PEP instances? | Enables end-to-end audit trail for complex workflows |
6. **Offline PEP operation**: Should there be a specification for PEP behaviour in disconnected/offline scenarios (e.g., edge deployments)? Stateless mode partially addresses this, but budget and revocation checks are unavailable. | Relevant for edge computing and air-gapped environments |

Appendix F: Reference Documents

| Document | Relevance |

|----------|----------|

| GiFo-RFC 0110 — GAuth Protocol Engine | Transport, routing, service mesh — the runtime environment for PEP deployment |

| GiFo-RFC 0111 — GAuth Authorization Framework | Authorization flows, roles, trust model — the policy framework the PEP enforces |

| GiFo-RFC 0115 — Three-Layer Capability Framework | Conceptual schema for PoA capabilities and governance profile ceiling tables — the capability model the PEP evaluates |

| GiFo-RFC 0116 v2.2 — GAuth Interoperability | Token format, credential schema, validation modes, error response schema — the credential contract the PEP validates and enforces |

| XACML 3.0 (OASIS) | Policy Enforcement Point architecture reference — terminology and patterns |

| RFC 9457 — Problem Details for HTTP APIs | Error response pattern reference for PEP error reporting |

| OpenPolicy Agent (OPA) | Reference PDP architecture — evaluation pipeline and decision structure patterns |

Disclaimer: ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN “AS IS” BASIS AND THE CONTRIBUTOR, THE ORGANIZATION THEY REPRESENT OR ARE SPONSORED BY (IF ANY), THE GIMEL FOUNDATION, AND ANY APPLICABLE MANAGERS OF ALTERNATE DOCUMENT STREAMS, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

* * *